

第 14 讲：Repo Map、AST、LSP：代码库结构如何变成 Agent Context

Coding Agent Notes (June 2026)

本讲在系列中的位置：#13 讲了 context engineering 的通用原则；本讲只讲代码库结构信息如何被抽取成 agent context。

已知前提：读者已经知道 context budget、retrieval、tool state 和 evidence 的区别。

本讲新增：repo map、AST / Tree-sitter、LSP 三类静态代码智能的边界、用途和失败模式。

读完后应该能回答：为什么 grep 不够；为什么 repo map 有用但不是全知图谱；AST 和 LSP 的差别是什么；agent 在不同任务中应该调用哪一种结构化代码信息。

1 核心图景

Coding agent 读 repo 时，最朴素的动作是搜索和打开文件。但真实代码库不是平铺文本：它有文件边界、函数、类、调用、类型、符号、diagnostics、构建配置和运行时状态。代码智能的任务，是把

这些结构转成能放进上下文的高信号信息。



这个 ladder 可以这样读：text search 找到候选字符串；repo map 给模型一个全局结构摘要；AST 提供语法级局部结构；LSP 提供 editor/language-server 语义服务；runtime evidence 通过测试、日志和执行确认静态判断。越往右，结构越强，但 setup、成本和失败模式也更多。

Insight. Repo intelligence 不是让 agent “理解整个代码库”，而是帮助它用更少 token 找到更可能相关的文件、符号和验证路径。它是 context selection over code structure。

1.1 例子：修一个 API 调用错误

假设用户说：“这个 endpoint 调用新版 payment API 后失败了。”一个只会 grep 的 agent 可能搜索 ‘payment’，打开几十个文件，然后在上下文里迷路。结构化 code intelligence 会让流程更清楚。

阶段	结构化问题	合适工具
候选定位	payment API 在哪些模块暴露？	repo map / grep
入口确认	endpoint handler 调了哪些函数？	AST / LSP definition
影响面	哪些调用点还用旧签名？	LSP references / grep
修改边界	哪些函数参数和返回值变化？	AST function boundary
验证	哪些测试覆盖 endpoint？	grep + runtime tests

这说明同一个任务会穿过多个层次。Repo map 给全局导航，AST 给局部结构，LSP 给跨文件语义，runtime evidence 给最终确认。任何一层单独使用都不够；关键是让 agent 知道每一层回答什么问题。

2 为什么 Grep 不够

Text search 的优点是简单、透明、跨语言。用户搜 ‘login’、‘auth’、‘UserService’，很快能找到候选文件。但 grep 不知道这个字符串是函数定义、调用、注释、测试 fixture 还是文档；也不知道符号是否被导出、是否被重命名、是否在 dead code 里。

问题	grep 能做什么	grep 做不到什么
找名字	匹配字符串	区分定义和调用
找文件	高召回候选	判断架构重要性
看历史	配合 git grep	理解类型和引用
调试	找错误文本	追踪执行路径

因此 grep 最适合作为第一轮 candidate generation，而不是最终 grounding。一个 agent 可以先用 grep 找候选，再用 repo map 判断全局位置，用 AST 抽取函数边界，用 LSP 找定义/引用，最后用测试或日志验证。

3 Repo Map

Aider 的 repo map 是最清楚的产品例子。它用一个 concise map 给 LLM 提供整个 git repo 的代码上下文。文档说明，repo map 可以包含 files、classes、methods、function signatures，让模型即使没有读取完整文件，也能看到 repo 里有哪些可用 API 和结构。



Repo map 的关键不是“把 repo 总结一遍”，而是在 token budget 下选择最有用的结构片段。Aider 文档提到，较大 repo 中 repo map 本身也可能太大，因此会用 graph ranking algorithm 在文件依赖图上选择部分 map，并通过 ‘-map-tokens’ 控制预算。默认约 1k tokens 的设置说明：repo map 是 context budget 内的结构提示，而不是完整知识库。

Repo map 元素	给 agent 的价值	风险
文件列表	建立项目地图	大 repo 中仍可能太长
函数签名	判断 API 怎么调用	看不到实现细节
类和方法	建立对象边界	动态行为不可见
排名/预算	把重要结构放进 context	ranking 可能错过冷门关键文件

Repo Map 是导航，不是证据。

Repo map 告诉 agent 哪里可能相关，不能替代打开文件和运行验证。它适合帮助 agent 决定下一步读什么，不适合直接作为修复正确性的依据。

3.1 怎样读 Repo Map

Repo map 最有用的读法，是把它当成“压缩后的代码库目录 + 符号索引”。读者或 agent 看到一个文件名、几个 class、几个 function signatures，就能形成候选路径：这个模块可能定义 API，那个模块可能使用 API，另一个模块可能是测试。

但是 repo map 省略了实现细节。一个函数签名告诉你可以调用什么参数，不告诉你函数内部是否有副作用；一个 class 名告诉你对象边界，不告诉你状态机；一个文件排名告诉你可能重要，不告诉你当前 bug 是否真的经过这里。

看到的 map 信息	可以推断	必须继续验证
文件路径	所属模块和大致责任	文件当前内容
函数签名	可用 API 形状	实现、副作用、异常
class / method	对象组织方式	状态和继承细节
排名较高	结构上更可能相关	bug 是否真的经过这里

3.2 Token Budget 是核心设计

Repo map 的难点不是生成，而是裁剪。小 repo 可以把大部分结构都放进 prompt；大 repo 必须选择。Aider 用 ‘-map-tokens’ 控制 repo map 预算，并且根据聊天状态动态调整大小。这其实是 #13 context budget 的具体应用：在“读全局结构”和“给当前文件/日志留空间”之间平衡。

一个好的 repo map 系统至少要记录三件事：当前 map 用了多少 token；哪些文件或符号被选入；为什么这些内容比其他候选更重要。否则 agent 无法判断 map 的缺口，也无法知道是否需要进一步搜索。

4 AST / Tree-sitter

AST 是 source code 的语法结构树。和 grep 不同，AST 能告诉你这是函数定义、调用表达式、参数列表、import、class body，还是注释。它适合做局部结构抽取：函数边界、签名、嵌套范围、语法节点、定义位置。

Tree-sitter 的价值在于它是面向编辑器和工具场景的 parser system，能够为多语言代码产生 syntax tree，并支持增量解析。对 coding agent 来说，Tree-sitter 常被用来抽取 symbols、tags、definitions、references-like patterns，或者把文件切成更合理的上下文块。

AST 能回答	例子	不能保证
语法范围	这个函数从哪到哪	运行时是否会执行
结构类型	这是 class、method、call	动态派发真实目标
局部依赖	import、decorator、参数	跨项目语义正确性
安全切片	按函数而非行切 context	业务含义

AST 特别适合 context chunking。按固定 token 或固定行数切文件，很容易把函数切断；按 AST 节点切，可以保证一个函数、类或方法作为完整语义单元进入上下文。

4.1 AST Query 和 Chunking

Tree-sitter 的实用价值不只在于“有一棵树”，还在于可以对树做 query。比如抽取所有 function definitions、class declarations、imports、decorators、call expressions。对 agent 来说，这些 query 可以把一个长文件转成结构化候选：

AST query	Agent 可得到	用途
definitions	函数/类名和范围	生成 repo map / tags
imports	依赖方向	判断模块关系
calls	调用表达式	找可能影响面
decorators	框架注册信息	找路由、测试、权限
errors	parse problem 节点	警告结构不可信

Chunking 时，AST 能避免两类错误。第一是切断函数，使模型只看到前半段逻辑。第二是混合多个不相关函数，使上下文看似丰富但决策噪声很高。按 AST 节点切块后，agent 可以先读函数签名和注释，再按需要展开函数体。

4.2 AST 的语义边界

AST 只是语法。它能告诉你 `foo.bar(x)` 是一个 member call，却不一定知道 `foo` 的运行时类型；它能看到 decorator，却不一定知道框架如何解释 decorator；它能看到 import，却不一定知道 bundler、dependency injection 或 monkey patch 改变了什么。因此 AST 是结构化阅读，不是最终语义理解。

5 LSP

LSP 是另一层抽象。Language Server Protocol 定义了 client 和 language server 之间的 JSON-RPC 消息。编辑器可以请求 hover、completion、document symbols、workspace symbols、diagnostics、semantic tokens、definition、references 等服务。协议还通过 capabilities 让 client/server 协商支持哪些功能。

对 coding agent 来说，LSP 的价值在于它复用了 IDE 生态里的语义能力。一个配置正确的 language server 通常比单纯 AST 更懂项目：它可能知道类型、跨文件引用、workspace symbols、diagnostics 和重命名影响范围。

LSP 功能	Agent 用法	风险
Definition	从调用跳到定义	服务器或索引不完整
References	找影响面	动态语言和反射可能漏
Diagnostics	获取错误和类型信息	依赖环境配置
Hover	获取类型/文档摘要	可能过短或过旧
Workspace symbols	全局找符号	符号命名噪声

LSP 不等于真相。协议只是边界，具体质量取决于 language server、项目配置、依赖是否安装、文件是否同步、是否是 generated code、宏和动态语言特性是否可解析。

5.1 LSP 请求生命周期

LSP 的一个重要细节是状态同步。Specification 里有 didOpen、didChange、didClose 等 document sync 概念。Client 和 server 必须在同一个文档状态上交流，否则 definition、diagnostics、references 都可能过期。对于 agent，这意味着：如果 agent 修改了文件，却没有刷新 LSP state，后续语义查询可能基于旧代码。

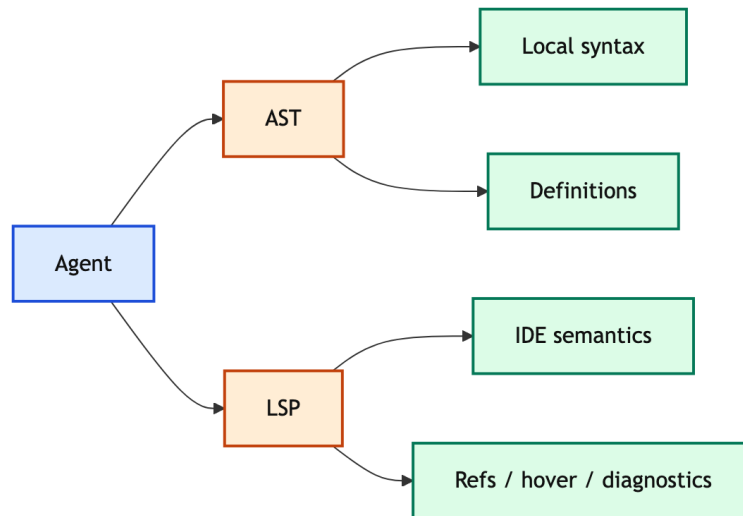
LSP 阶段	发生什么	Agent 风险
Initialize	client/server 交换 capabilities	以为所有功能都可用
Sync document	打开、修改、关闭文档	查询基于旧内容
Request feature	hover/definition/references	结果可能 partial 或 stale
Handle response	应用结果到上下文	未检查 null/error
React to change	文件或项目状态变化	旧结果仍被复用

这也是为什么 LSP 结果应该带 provenance：使用哪个 server，在哪个 workspace root，基于哪个 file version，是否有 diagnostics，是否返回 null。没有这些元信息，agent 很难判断语义查询的可信度。

5.2 Capability 不是能力保证

LSP 中 capabilities 表示 client/server 支持某个 feature，但支持不等于高质量。一个 server 可能支持 references，但对宏、动态 import 或 generated code 表现不好；也可能需要项目依赖安装后才有完整语义。因此 agent 不应该把 LSP 结果当作无条件事实，而应该把它看成强 evidence source 之一。

6 AST vs LSP



简单说，AST 更像“这个文件的语法结构”，LSP 更像“IDE 看到的项目语义服务”。AST 通常更轻、更可控、更适合本地切片；LSP 更强但依赖服务器、项目配置和语言生态。

维度	AST / Tree-sitter	LSP
粒度	文件内语法节点	项目级语言服务
依赖	grammar / parser	language server + project config
速度	通常轻量	可能需要索引和启动
语义	结构强，语义弱	语义更强但不稳定
适合	chunking、tags、签名抽取	definition、references、diagnostics

7 Agent 应该问什么

选择代码智能 primitive 时，不要先问“哪个最强”，而要问当前问题需要什么证据。要找候选文件，用 grep 或 repo map；要切函数，用 AST；要找跨文件引用，用 LSP；要证明修复正确，用测试和日志。

Agent 问题	推荐工具	下一步
这个符号在哪定义？	LSP definition / AST tags	打开定义文件
这个 API 怎么用？	repo map + references	读取调用示例
这个文件太长怎么切？	AST chunking	取函数/类块
改动影响哪里？	LSP references + grep	排序风险路径
测试为什么失败？	runtime logs/tests	回到代码结构定位

弱充分原则：
用能回答当前问题的最弱结构化工具。grep 能回答就不要启动重索引；AST 能回答就不必依赖 LSP；静态信息能缩小范围，但最终行为仍要靠 runtime evidence 验证。

7.1 例子：重命名一个跨文件函数

假设任务是把 ‘createCharge’ 改成 ‘createPaymentIntent’，同时兼容旧调用。这个任务看起来简单，但如果只用文本替换，很容易误改注释、文档示例、测试 fixture 或不同语义的同名函数。

步骤	推荐结构信息	目的
确认定义	LSP definition / AST definition	找到真实 API owner
找调用点	LSP references + grep fallback	建立影响面
分类调用	production / tests / docs / examples	区分必须改和可延后
局部修改	AST chunk around call site	避免跨函数误改
验证	typecheck + targeted tests	确认静态修改有效

这里 LSP references 负责高精度影响面，但 grep 仍然有价值：它可以发现文档、字符串配置和动态调用。AST chunk 负责让 agent 只编辑调用所在函数，而不是整文件。Runtime evidence 负责最终确认新旧调用是否都还能工作。

这个例子也说明：结构工具之间不是替代关系，而是互相补缺。LSP 漏掉动态路径，grep 可以兜底；grep 太噪声，LSP 可以排序；AST 不懂跨文件语义，但能给编辑提供安全边界。

7.2 从结构查询到行动计划

结构化代码智能的产物应该转成行动计划，而不是直接转成 patch。比如 LSP references 找到 18 个调用点，agent 不应该立刻全部修改。它应该先分类：哪些是生产路径，哪些是测试，哪些是兼容层，哪些是 dead code；再选择最小改动范围；最后说明哪些调用点没有改以及为什么。

这一步把 #14 和 #13 连接起来：repo intelligence 负责产生候选结构，context engineering 负责选择哪些进入当前推理，harness/runtime 负责验证行动是否正确。

8 工具输出如何变成 Prompt Context

结构工具的原始输出通常不适合直接塞进 prompt。一个 LSP references response 可能包含几十个 URI 和 range；一个 AST tree 可以有几百行 S-expression；一个 repo map 可能有很多文件。Agent 需要的是 task-specific rendering。

原始输出	不应直接放入	更好的 prompt 形态
AST tree	全量节点 dump	函数签名、范围、关键子节点
References	未排序 URI 列表	按 production/test/docs 分类
Diagnostics	原始 JSON	错误、位置、原因、相关符号
Repo map	全 repo 文本	当前任务相关 top subset
Grep result	所有匹配行	去重后的候选文件和理由

Prompt context 应该保留 provenance。比如每条 reference 应有 path、line、kind、source tool、timestamp 或 file version。这样 agent 在后续写 summary 或 patch 时，能说明自己的依据来自哪里，也便于人类 review。

结构化 context 最小字段：
source tool； path； line/range； symbol； why relevant； confidence； next action。

9 失败模式

第一，index stale。repo map、AST cache、LSP index 可能落后于当前 diff。agent 读到旧符号后会编辑错误位置。第二，parse failure。语法错误、宏、模板、generated code、混合语言文件都可能让 parser 输出不完整结构。第三，LSP misconfiguration。依赖没装、workspace root 错、language server 崩溃，会导致 diagnostics 或 references 不可信。

第四，semantic gap。AST 和 LSP 都不能完整理解运行时行为。反射、依赖注入、配置驱动路由、SQL、模板、前端 build pipeline、feature flags，都可能让静态结构和真实执行路径分离。

失败	表现	缓解
Stale map	推荐旧文件或旧签名	每轮改动后刷新或标记版本
Bad parse	chunk 边界错误	回退到文本窗口和测试
Bad LSP setup	definitions/references 漏失	检查 workspace/root/deps
Dynamic behavior	静态路径不等于执行路径	加 runtime evidence
Overtrust	agent 不打开原文件就修改	要求 evidence chain

10 落地 Checklist

把 repo intelligence 接入 agent context 时，可以按四步做。第一，先提供 cheap overview：文件树、README、repo map、主要目录。第二，基于任务提出候选符号和文件，不要全量展开。第三，用 AST 或 LSP 获取精确边界、定义、引用和 diagnostics。第四，把静态发现转化为可验证计划：打开哪些文件、改哪些 hunk、跑哪些测试。

阶段	输出	验收信号
Overview	相关目录和候选文件	不超过一页
Structure	符号、签名、函数边界	可定位到路径/行号
Semantics	定义、引用、diagnostics	能解释影响面
Evidence	测试、日志、diff	能验证静态判断

11 结构化上下文设计模式

把 repo intelligence 喂给 agent 时，不应该只给一个长列表。更好的做法是把结构化信息做成几个固定模式。

模式	内容	适合任务
Symbol card	名称、路径、签名、定义范围	API 使用和定位
Impact card	被谁调用、依赖谁、测试在哪里	修改影响分析
Chunk card	一个函数/类的完整源码块	局部修复和重构
Diagnostic card	LSP error、位置、相关类型	类型错误和编译失败
Runtime card	命令、输出、失败栈	验证静态假设

这些 card 的价值，是把工具输出变成可审查上下文。原始 LSP response 可能很长，原始 AST tree 可能不可读，grep 结果可能重复。Agent 真正需要的是：这个符号是什么，在哪里，为什么相关，下一步应该读什么或验证什么。

例如 Symbol card 可以这样写：PaymentClient.createCharge(path: services/payment/client.py, line 42, callers: checkout.py, billing.py, tests: tests/payment/test_client.py)。这比把整棵 AST 塞进 prompt 更有用，因为它直接连接了定义、调用者和测试入口。

12 Evidence Card 的最小格式

为了让结构化工具输出真正进入 agent reasoning，可以把 repo map、AST、LSP 和 runtime 输出统一成 evidence card。它不是新的大抽象，而是一个固定字段表，让 agent 和 reviewer 都能看懂“这个上下文为什么出现”。

字段	内容	作用
Claim	这段证据支持什么判断	防止只列文件不说明意义
Source	repo map / AST / LSP / runtime	标明信号来源
Location	文件、行号、symbol、命令	可追溯
Confidence	高/中/低和原因	防止过信静态工具
Next check	下一步打开、运行或验证什么	连接行动

例如，LSP references 返回十个调用点时，不应该直接把十个路径全部塞进 prompt。更好的 evidence card 是：“PaymentClient.createCharge 在 checkout 和 billing 两条路径中被调用；当前 issue 涉及 checkout 失败，因此先打开 checkout.py 和对应测试；billing 路径作为 regression candidate。”这把无序引用列表变成了可执行的验证计划。

Evidence card 还能降低工具冲突。若 repo map 推荐一个文件，LSP 没找到引用，runtime trace 又显示请求进入另一个 handler，card 会把信号来源并列放出来，让 agent 知道需要验证，而不是选择性相信某一个工具。

13 怎样评估 Repo Intelligence

Repo intelligence 的评估也不能只看最终 patch 是否成功。它应该单独回答：候选文件找得准不准，符号边界是否正确，引用是否漏掉，静态输出是否帮助减少 token 和工具调用，agent 是否更快找到验证路径。

指标	问题	失败信号
Recall	相关文件是否被候选集覆盖	真正文件没被列出
Precision	候选是否过多噪声	agent 读了大量无关文件
Boundary	chunk 是否完整	函数被切断或混杂
Freshness	index 是否跟当前 diff 一致	agent 引用旧签名
Utility	是否减少探索成本	工具输出变多但决策没变

最实用的离线测试，是给定一个 bug-fix 任务，只评估工具能否把真正相关文件排到前面，而不让模型生成 patch。这样可以把 context finding 和 patch generation 分开。后面的 ContextBench / SWE-ContextBench 会更系统地讲这个问题；在本讲里，先记住一个原则：如果结构工具不能改善“找上下文”，它就不应该被引入 agent pipeline。

14 实践选择树

把结构化代码智能用于 agent 时，可以用下面的选择树：

不知道相关文件：先 grep / repo map。
 知道文件但文件太长：用 AST 切函数或类。
 知道符号但不知道影响面：用 LSP definition / references。
 知道修改点但不确定行为：运行 targeted test / typecheck。
 工具结果互相矛盾：回到原始文件和 runtime evidence。

这个选择树能避免两个极端。一个极端是“只 grep”：所有问题都变成文本搜索，agent 会读大量无关文件。另一个极端是“只信 IDE 语义”：一旦 LSP 配置不完整，agent 会把缺失结果误认为没有影响面。真正稳的流程是在多个层次之间交叉验证。

任务意图	第一工具	第二工具
理解模块 API	repo map	打开文件 / examples
修类型错误	LSP diagnostics	typecheck
重构函数	LSP references	AST chunking
定位 bug 路径	grep / repo map	runtime trace
写测试	repo map + examples	test runner

15 静态信息到 Runtime Evidence

代码结构能缩小搜索空间，但不能证明行为。一个 function signature 看起来正确，测试可能仍然失败；LSP references 看起来完整，动态路由可能仍然漏掉；AST chunk 看起来局部清楚，真实 bug 可能来自配置。

因此，agent 应该把静态发现转换成 runtime questions：

静态发现	Runtime question	验证方式
找到 handler	请求是否真的进这里？	trace/log/manual request
找到调用点	旧签名是否仍被执行？	targeted test
找到类型错误	编译器是否同意？	typecheck
找到可疑函数	修复是否改变失败输出？	reproduction test
找到影响面	是否破坏其他路径？	regression tests

这条 handoff 很关键。没有 runtime evidence，repo map、AST、LSP 只是帮助 agent 更快形成假设；有了 runtime evidence，结构化假设才变成可审查的修复链路。

16 排序策略

结构化上下文还需要排序。Agent 不可能同时阅读所有候选文件、所有 symbol、所有 references。排序策略应该把“任务相关性”和“验证价值”结合起来，而不是只按文本相似度或 PageRank。

排序信号	为什么有用	风险
Issue keyword match	快速找到表面相关模块	容易被日志或文档误导
Call graph distance	找到调用链附近代码	动态调用可能漏掉
Test proximity	连接失败测试和实现	测试覆盖不足时偏斜
Recent diff / blame	找到近期变化	旧 bug 不一定在新改动中
Runtime trace	贴近真实执行路径	需要可复现环境

一个稳健的 agent 不应该把排序当作最终事实。排序只是决定先看什么；真正的判断仍然要回到原始文件、失败测试和 runtime evidence。最好的 repo intelligence 输出，会同时给出 top candidates 和“为什么排在前面”，并明确哪些低排名项仍可能重要。

17 索引失效治理

Repo map、AST cache 和 LSP workspace 都会失效。代码被修改后，旧索引可能继续推荐已经不存在的签名；依赖没安装好时，LSP 可能漏掉 references；多 worktree 并行时，不同 workspace 的 index 可能指向不同版本。Agent 如果不知道索引版本，就会把 stale signal 当成事实。

失效来源	表现	治理方式
Dirty diff	index 和当前文件不一致	修改后局部刷新
Wrong root	LSP 以错误 workspace 启动	记录 root 和 config
Missing deps	diagnostics / refs 不完整	暴露 setup 状态
Parallel worktree	不同 agent 看到不同版本	index 绑定 worktree id
Generated code	map 指向生成文件或旧产物	标注 generated / ignored paths

因此，结构化上下文应该带 metadata：生成时间、commit/worktree、工具版本、workspace root、是否包含 dirty diff、是否成功加载依赖。没有这些 metadata，repo intelligence 看似精确，实际不可审计。对 agent 来说，最危险的不是不知道，而是拿着过期索引自信地修改代码。

18 工具不可用时的降级路径

真实仓库里，AST parser、LSP、测试环境都可能不可用。Agent 系统不能因为某个高级工具失败就停住，也不能把失败静默吞掉。更稳的做法，是为每个结构化工具准备降级路径，并把降级事实写进 evidence。

失败工具	降级路径	必须告知 agent
Repo map 过期	重新扫描文件树 + grep 入口	map 不可信，候选只是粗略
AST parse 失败	按函数名/缩进/测试片段打开原文	chunk 边界可能不精确
LSP 不启动	grep references + typecheck / tests	references 可能漏掉
Diagnostics 缺失	编译器或 test runner 输出	IDE 语义不可用
Runtime 不可复现	静态证据 + 明确未验证范围	patch 需要人工 review

降级路径的关键是诚实。一个 agent 如果知道“LSP 未启动，所以 references 不完整”，它会更谨慎地打开调用点和测试；如果系统只返回空 references，agent 可能误判“没有影响面”。因此，工具失败本身也是上下文，必须进入 trajectory 和 final evidence。

从产品角度看，降级路径还决定用户信任。用户不要求 agent 每次都有完美工具，但需要知道当前结论来自强信号还是弱信号。强信号可以支持自动 patch；弱信号更适合生成调查报告、候选路径或人工 review checklist。

19 局限与开放问题

Repo map 可能错过低频但关键的文件。AST 可能无法表达类型、项目配置和跨语言边界。LSP 可能依赖复杂环境，且不同语言服务器质量差异很大。对前端、配置、数据库、构建系统和动态语言来说，静态结构尤其不够。

因此，repo intelligence 应该和 #13 的 context engineering 结合：先用结构缩小搜索空间，再用原始文件和 runtime evidence grounding。不要把结构化工具的输出当作最终事实。

20 最终 Takeaway

Repo map、AST、LSP 的共同作用，是把大代码库压缩成 agent 能行动的结构化上下文。Repo map 提供全局导航，AST 提供局部语法，LSP 提供语义服务，runtime evidence 提供最终验证。

一个好的 coding agent 不应该只会 grep，也不应该盲信最强索引。它应该根据当前问题选择最弱充分工具，把结构信号转成可审查的 evidence chain。

参考资料

- [1] [Aider Repository Map](#). concise repo map, signatures, graph ranking, and token budget
- [2] [Tree-sitter Using Parsers](#). parser APIs, syntax trees, and language-neutral parsing concepts
- [3] [Language Server Protocol 3.17](#). JSON-RPC protocol boundary, capabilities, document sync, and language features